

Multicore Programming Models

**Fu Jen Catholic University,
Hsin Chuang, Taipei, Taiwan.**

周賜福

輔仁大學,資工學系

Graduate Course – 2020-2021 First Semester



Outline

- **What is Multicore (Parallel) architecture?**
- **Why study Multicore Architecture?**
- **Why study it today?**
- **Current Application Trend**
- **Engineering and Scientific Demand**
- **What about Commercial computing?**
- **Application and Architectural Trend**
- **How do we exploit?**
- **Different Types of Models that are currently in Use**

What is Multicore (Parallel) Architecture?

A parallel computer is a collection of processing elements that cooperate to solve large problems fast

Some broad issues:

- **Resource Allocation:**

- how large a collection?
- how powerful are the elements?
- how much memory?

- **Data access, Communication and Synchronization**

- how do the elements cooperate and communicate?
- how are data transmitted between processors?
- what are the abstractions and primitives for cooperation?

- **Performance and Scalability**

- how does it all translate into performance?

– how does it scale?

Why Study Multicore or Parallel Architecture?

- **What is the Role of a computer architect:**
- To design and engineer the various levels of a computer system to maximize **performance** and **programmability** within limits of **technology** and **cost**.

Parallelism:

- Provides alternative to faster clock for performance
- Applies at all levels of system design



Why Study it Today?

History: diverse and innovative organizational structures, often tied to novel programming models

Rapidly maturing under strong technological constraints

- The “killer micro” is ubiquitous
- Laptops and supercomputers are fundamentally similar!
- Technological trends cause diverse approaches to converge

Technological trends make parallel computing inevitable

- In the mainstream

Need to understand fundamental principles and design tradeoffs, not just taxonomies

- Naming, Ordering, Replication, Communication performance

The history of Microprocessor

- **First Microprocessor(1970's)**
 - Intel 4004
- **PC spreads in the world(1980's)**
 - Up to 32bits microprocessor
 - AMD followed Intel's technology

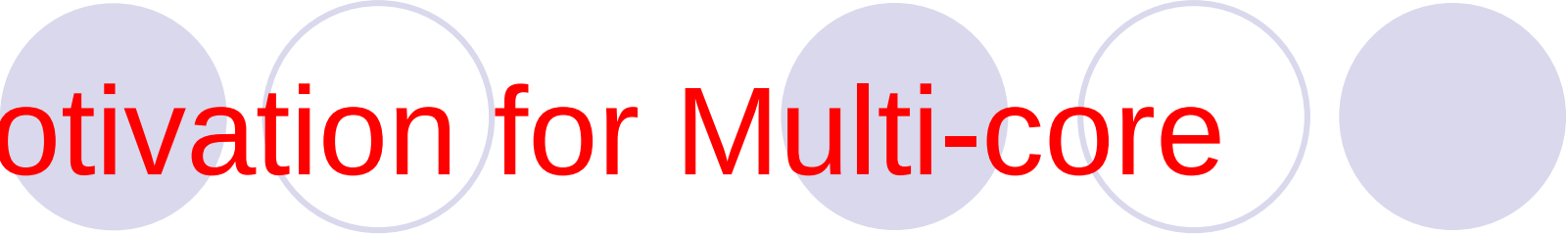
The history of Microprocessor

■ **Flood of Computer Tasks(1990's)**

- Increasing of Computer user
 - Server management → Construct the database
 - We need better performance of PC or Server.
- These demands accelerate the development of microprocessor.

■ **Emergence of Multi-core Processor(2000's)**

- Limits of improvement of single core
- Turn over of the idea that improve the MP technology
 - Put execution cores in one die



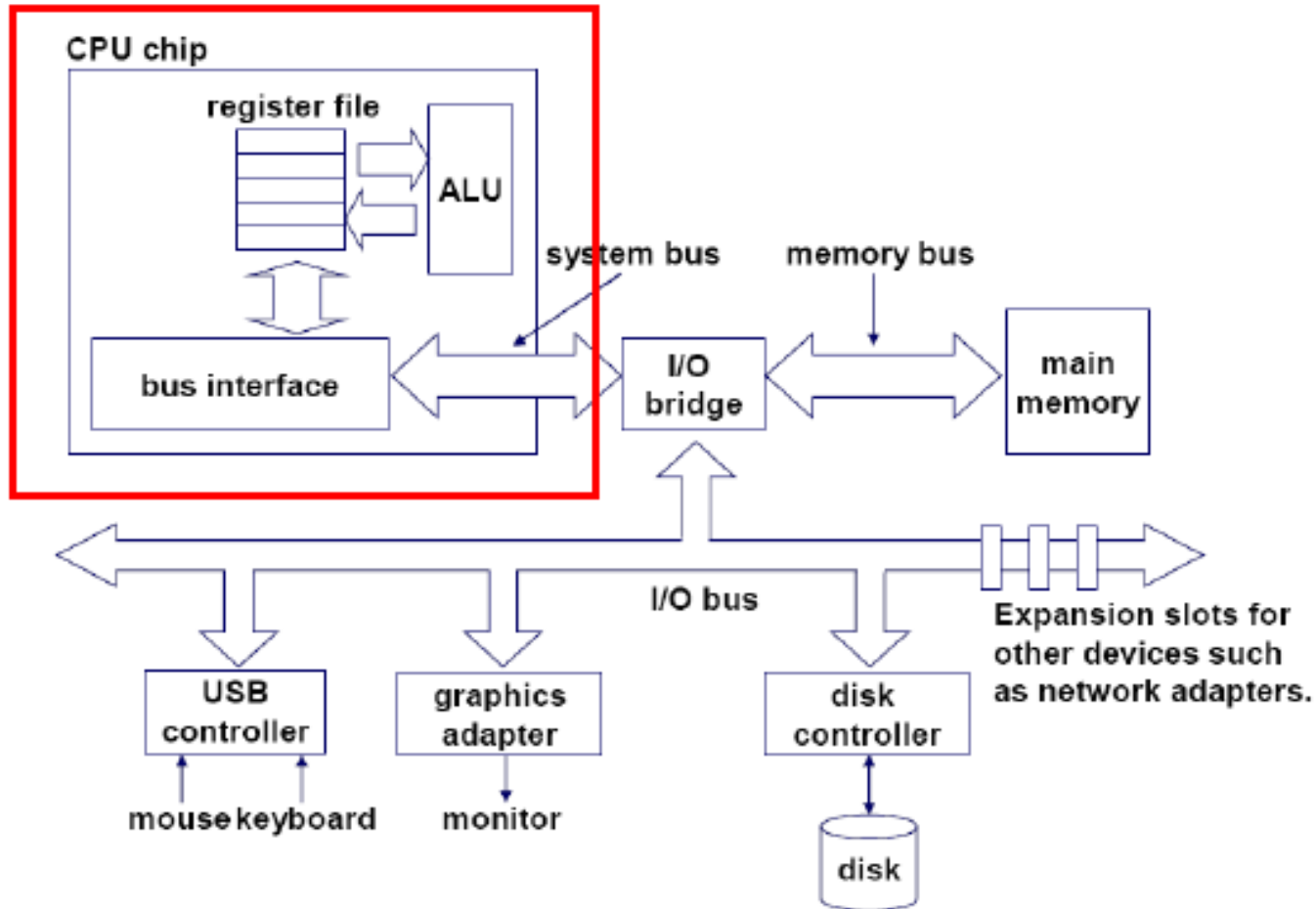
Motivation for Multi-core

- Exploits increased feature-size and density
- Increases functional units per chip (spatial efficiency)
- Limits energy consumption per operation
- Constrains growth in processor complexity

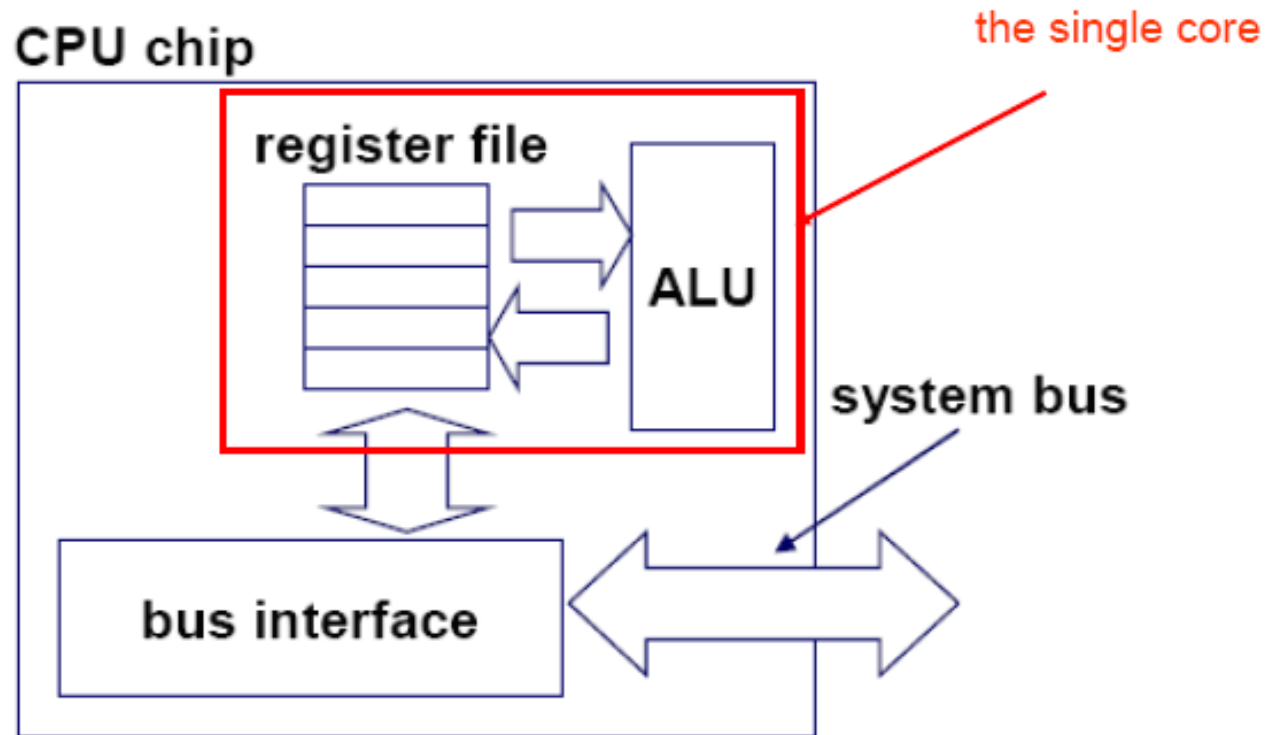
Multi-core Computers

- A multi-core processor is a processing system composed of two or more independent cores (or CPUs). The cores are typically integrated onto a single integrated circuit die (known as a chip multiprocessor or CMP), or they may be integrated onto multiple dies in a single chip package.
- A many-core processor is one in which the number of cores is large enough that traditional multi-processor techniques are no longer efficient - this threshold is somewhere in the range of several tens of cores - and likely requires a network on chip.

Single core Computers

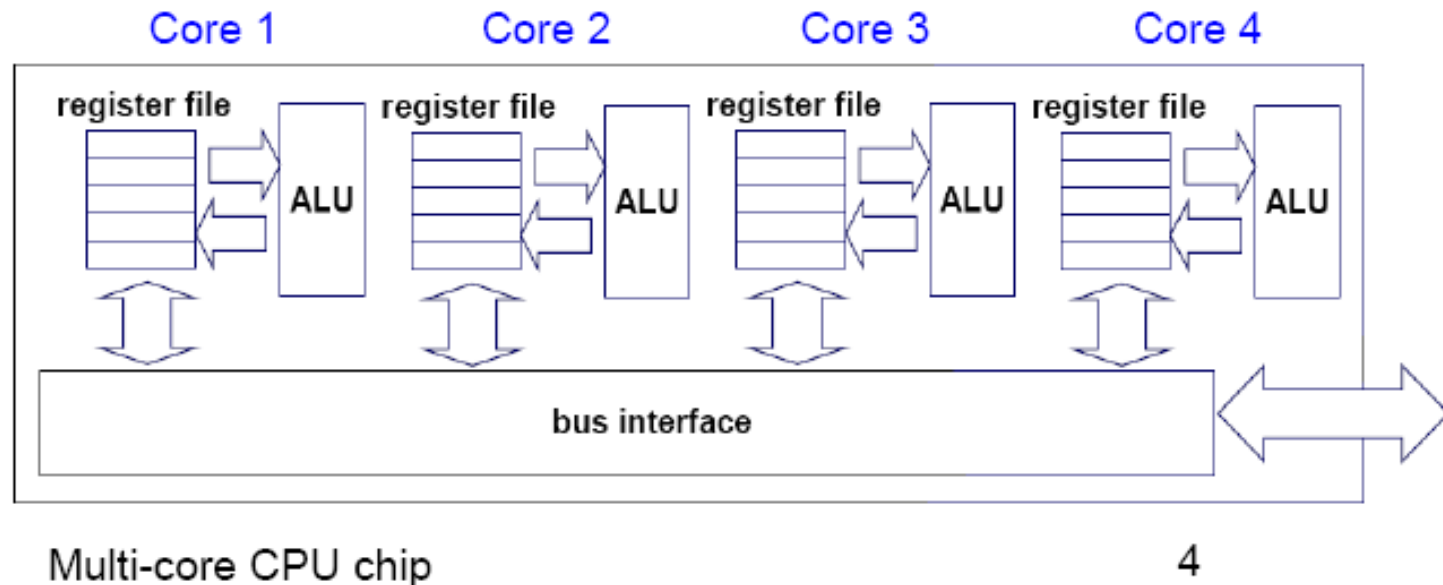


Single core CPU chip



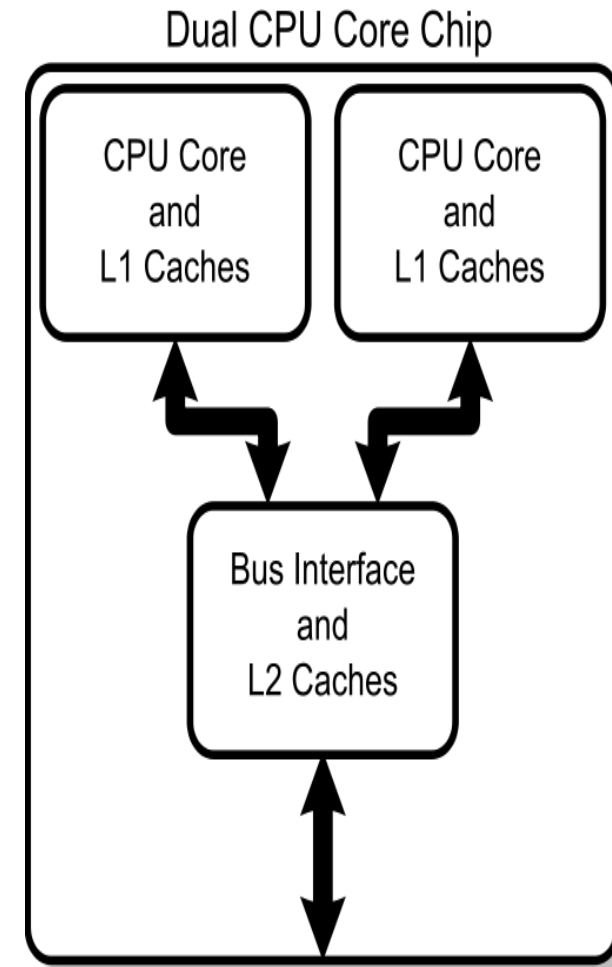
Multi-core Architectures

This lecture is about current Multi-core trend.
Replicate multiple processor on a single die.



Current Multi-core Computers

- **dual-core processor** contains two independent microprocessors.
- **A dual core** set-up is somewhat comparable to having multiple, separate processors installed in the same computer, but because the two processors are actually plugged into the same socket, the connection between them is faster. Ideally, a dual core processor is nearly twice as powerful as a single core processor. In practice, performance gains are said to be about fifty percent: a dual core processor is likely to be about one-and-a-half times as powerful as a single core processor.



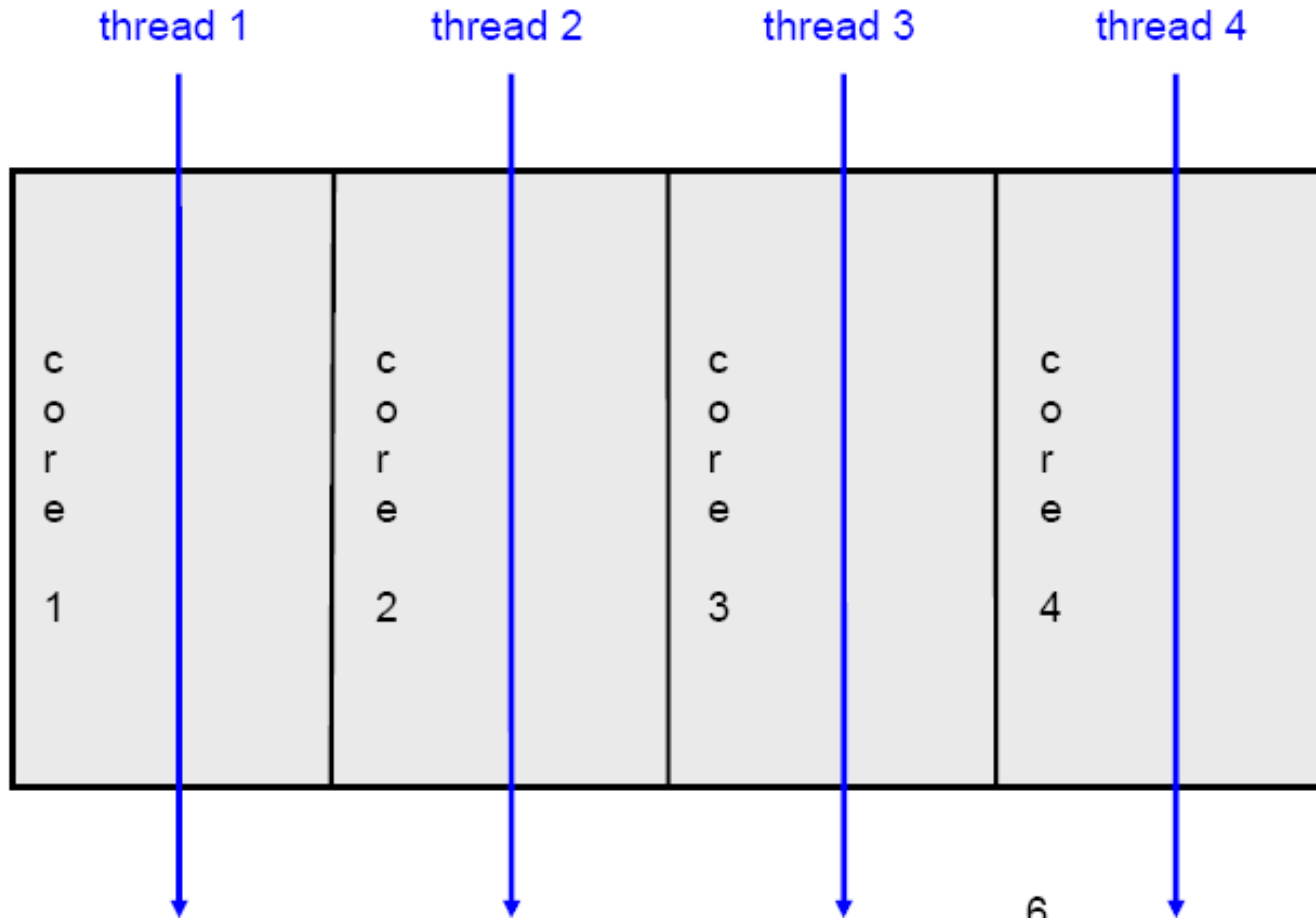
Current Multi-core Computers

- A multi-core processor implements multiprocessing in a single physical package. Cores in a multi-core device may be coupled together tightly or loosely. For example, cores may or may not share caches, and they may implement message passing or shared memory inter-core communication methods. Common network topologies to interconnect cores include: bus, ring, 2-dimensional mesh, and crossbar.
- All cores are identical in *symmetric* multi-core systems and they are not identical in *asymmetric* multi-core systems. Just as with single-processor systems, cores in multi-core systems may implement architectures such as superscalar, vector processing, or multithreading.

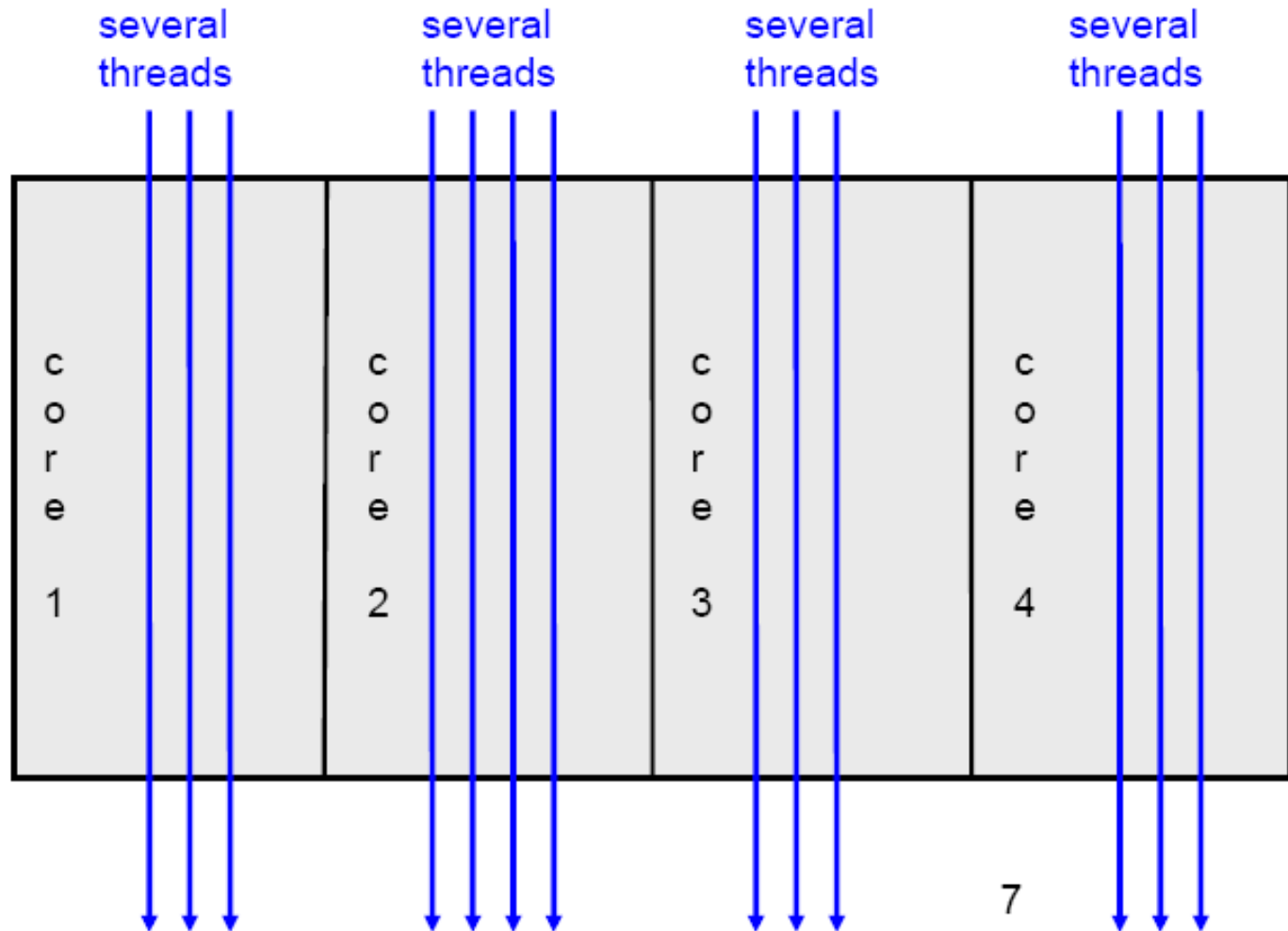
Current Multi-core Computers

- **Multi-core processors** are widely used across many application domains including: general-purpose, embedded, network, digital signal processing, and graphics.
- **The amount of performance** gained by the use of a multi-core processor is strongly dependent on the software algorithms and implementation.
- **Multi-core processing** is a growing industry trend as single core processors rapidly reach the physical limits of possible complexity and speed.
- Companies that have produced or are working on multi-core products include AMD, ARM, Broadcom, Intel, and VIA.

Cores run in Parallel



Within each core, threads are time-sliced (just like on a uniprocessor)





CMP Benefits

- with a shared on-chip cache memory, communication events can be reduced to just a handful of processor cycles.
- therefore with low latencies, communication delays have a much smaller impact on overall performance.
- threads can also be much smaller and still be effective.
- automatic parallelization more feasible.

Current Processors

Intel i3 Processor

Two or four cores

Intel i5 Processor

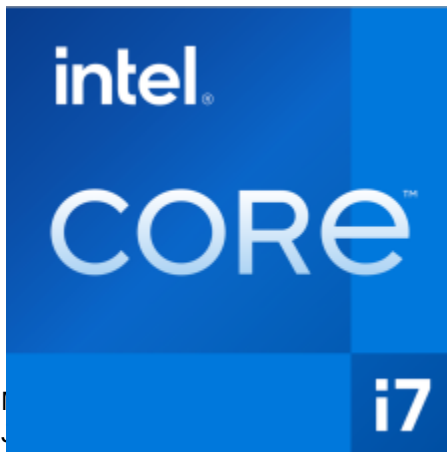
Two, four or six cores

Intel i7 Processor

Two, four, six or eight cores

Intel i9 Processor

**six, eight, Ten, twelve, fourteen
and Eighteen**



Intel® Core™ i7-1160G7 處理器（12M 快取記憶體，最高可達 4.40 GHz）

12 MB Intel® Smart Cache 快取記憶體

4 核心

8 執行緒

4.40 GHz 最大超頻

G - Includes discrete graphics on package

11th Generation

Intel i9 in particular has multithreading ability

Intel i9 processor

It has multithreading ability



Intel® Core™ i9-10900K 處理器

20 MB Intel® Smart Cache 快取記憶體

10 核心

20 執行緒

5.30 GHz 最大超頻

K - Unlocked

10th Generation

加入比較 無法提供價格

Inevitability of Multicore programming

- **Application demands:** Our insatiable need for cycles
 - • Scientific computing: CFD, Biology, Chemistry, Physics, ...
 - • General-purpose computing: Video, Graphics, CAD, Databases, TP...
- **Technology Trends**
 - • Number of transistors on chip growing rapidly
 - • Clock rates expected to go up only slowly
- **Architecture Trends**
 - • Instruction-level parallelism valuable but limited
 - • Coarser-level parallelism, as in MPs, the most viable approach
- **Economics**
- **Current trends:**
 - • Today's microprocessors have multiprocessor support
 - • Servers & even PCs becoming MP: Sun, SGI, CO
- **Tomorrow's microprocessors are multiprocessors**



Multi-Core Processor

- **Multi-core processors** are widely used across many application domains including: general-purpose, embedded, network, digital signal processing, and graphics.
- **The amount of performance** gained by the use of a multi-core processor is strongly dependent on the software algorithms and implementation.
- **Multi-core processing** is a growing industry trend as single core processors rapidly reach the physical limits of possible complexity and speed.
- Companies that have produced or are working on multi-core products include AMD, ARM, Broadcom, Intel, and VIA.

What applications benefit from Multi-core architectures?

- Database servers
- Web servers
- Telecommunication markets
- Multimedia applications
- Scientific applications
- In general, applications with Thread-level parallelism (as opposed to instruction-level parallelism)



Programming for Multi-core

- Programmers must use threads or processes.
- Spread the workload across multiple cores.
- Write parallel algorithms.
- OS will map threads/processes to cores



Current Application Trends

Demand for cycles fuels advances in hardware, and vice-versa

- **Cycle drives exponential increase in microprocessor performance**
- **Drives parallel architecture harder: most demanding applications**

Range of performance demands

- **Need range of system performance with progressively increasing cost**
- **Platform pyramid**

Final Goal is Speedup

Goal of any application in using Multicore or parallel architecture is Speedup

$$\text{Speedup (p processors)} = \frac{\text{Performance (P processors)}}{\text{Performance (1 processor)}}$$

For a fixed problem size (input data set), performance = 1/time

$$\text{Speedup}_{\text{fixed problem}} (\text{p processors}) = \frac{\text{Time (1 Processor)}}{\text{Time P processors}}$$

Finding Enough Parallelism

- Suppose only part of an application seems parallel
- Amdahl's law
 - let s be the fraction of work done sequentially, so $(1-s)$ is fraction parallelizable
 - P = number of processors

$$\text{Speedup}(P) = \text{Time}(1)/\text{Time}(P)$$

$$\leq 1/(s + (1-s)/P)$$

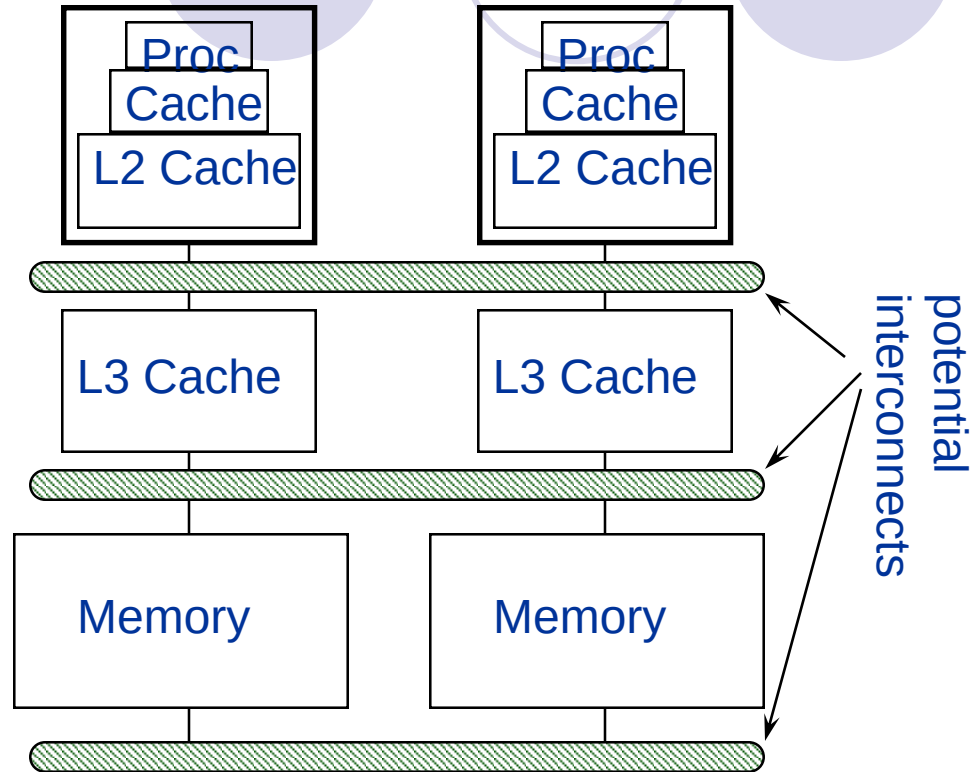
$$\leq 1/s$$

- Even if the parallel part speeds up perfectly performance is limited by the sequential part

Overhead of Parallelism

- Given enough parallel work, this is the biggest barrier to getting desired speedup
- Parallelism overheads include:
 - cost of starting a thread or process
 - cost of communicating shared data
 - cost of synchronizing
 - extra (redundant) computation
- Each of these can be in the range of milliseconds (=millions of flops) on some systems
- Tradeoff: Algorithm needs sufficiently large units of work to run fast in parallel (I.e. large granularity), but not so large that there is not enough parallel work

Conventional Storage Hierarchy



- Large memories are slow, fast memories are small
- Storage hierarchies are large and fast on average
- Parallel processors, collectively, have large, fast cache
 - the slow accesses to “remote” data we call “communication”
- Algorithm should do most work on local data



Load Imbalance

- **Load imbalance is the time that some processors in the system are idle due to**
 - insufficient parallelism (during that phase)
 - unequal size tasks
- **Examples of the latter**
 - adapting to “interesting parts of a domain”
 - tree-structured computations
 - fundamentally unstructured problems
- **Algorithm needs to balance load**

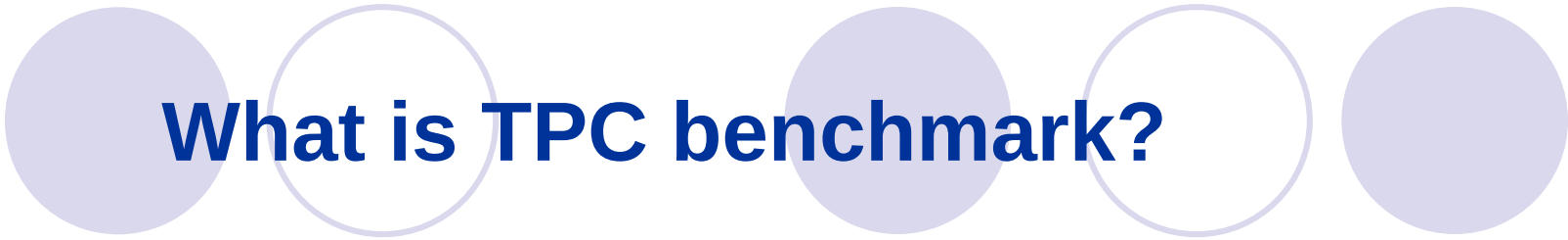
Engineering and Scientific Computing Demand

Large parallel machines a mainstay in many industries

- **Petroleum** (reservoir analysis)
- **Automotive** (crash simulation, drag analysis, combustion efficiency),
- **Aeronautics** (airflow analysis, engine efficiency, structural mechanics, electromagnetism),
- **Computer-aided design**
- **Pharmaceuticals** (molecular modeling)
- **Visualization**
 - in all of the above
 - entertainment (films like Toy Story)
 - architecture (walk-throughs and rendering)
- **Financial modeling (yield and derivative analysis)**
- **etc.**

What about Commercial Computing?

- **Also relies on parallelism for high end**
 - • Scale not so large, but use much more wide-spread
 - • Computational power determines scale of business that can be handled
- **Databases, online-transaction processing, decision support, data mining, data warehousing ...**
- **TPC benchmarks (TPC-C order entry, TPC-D decision support)**
 - • Explicit scaling criteria provided
 - • Size of enterprise scales with size of system
 - • Problem size no longer fixed as p increases, so throughput is used as a performance measure (transactions per minute or tpm)



What is TPC benchmark?

Transaction Processing Performance council

www.tpc.org

TPC defines transaction processing and database benchmark and delivers trusted results for industry

The current version of the TPC-C benchmark is Version 5.11. The TPC-C has continued to evolve its benchmarks to remain as representative of current practice as possible.

Summary of Application Trends

- Transition to parallel computing has occurred for
 - scientific and engineering computing
- In rapid progress in commercial computing
 - • Database and transactions as well as financial
 - • Usually smaller-scale, but large-scale systems also used
- Desktop also uses **multithreaded** programs, which are a lot like parallel programs
- Demand for improving **throughput** on sequential
- workloads
 - • Greatest use of small-scale multiprocessors
- Solid application demand exists and will increase



Architectural Trends

- Architecture translates **technology's gifts** to **performance** and **capability**
- Resolves the tradeoff between parallelism and locality
 - • Current microprocessor: 1/3 compute, 1/3 cache, 1/3 off-chip connect
 - • Tradeoffs may change with scale and technology advances
- Understanding microprocessor architectural trends
- • Helps build intuition about design issues or parallel machines
- • Shows fundamental role of parallelism even in “sequential” computers
- Four generations of architectural history: tube, transistor, IC, VLSI
 - • Here focus only on VLSI generation
- Greatest delineation in VLSI has been in **type of parallelism exploited**

Arch. Trends: Exploiting Parallelism

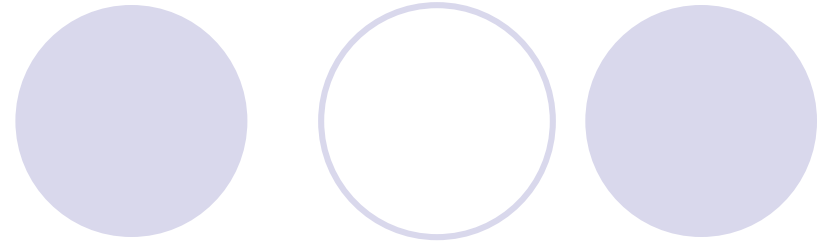
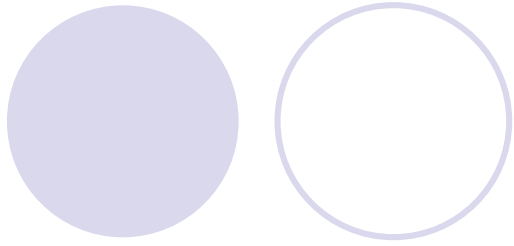
- Greatest trend in VLSI generation is increase in parallelism
- • **Up to 1985: bit level parallelism:** 4-bit -> 8 bit -> 16-bit
 - – slows after 32 bit
 - – adoption of 64-bit now under way, 128-bit far (not performance issue)
 - – great inflection point when 32-bit micro and cache fit on a chip
- • **Mid 80s to mid 90s: instruction level parallelism**
 - – pipelining and simple instruction sets, + compiler advances (RISC)
 - – on-chip caches and functional units => superscalar execution
 - – greater sophistication: out of order execution, speculation, prediction
 - » to deal with control transfer and latency problems
- • **Next step: thread level parallelism**

Economics of Multicore Architecture

- **Commodity microprocessors not only fast but CHEAP**
 - • **Development cost is tens of millions of dollars (5-100 typical)**
 - • **BUT, many more are sold compared to supercomputers**
 - • **Crucial to take advantage of the investment, and use the commodity building block**
 - • **Exotic parallel architectures no more than special-purpose**
- **Multiprocessors being pushed by software vendors (e.g. database) as well as hardware vendors**
- **Standardization by Intel makes small, bus-based SMPs**
- **commodity**
- **Desktop: few smaller processors versus one larger one?**
 - • **Multiprocessor on a chip**

Summary: Why Parallel (Multicore) Architecture?

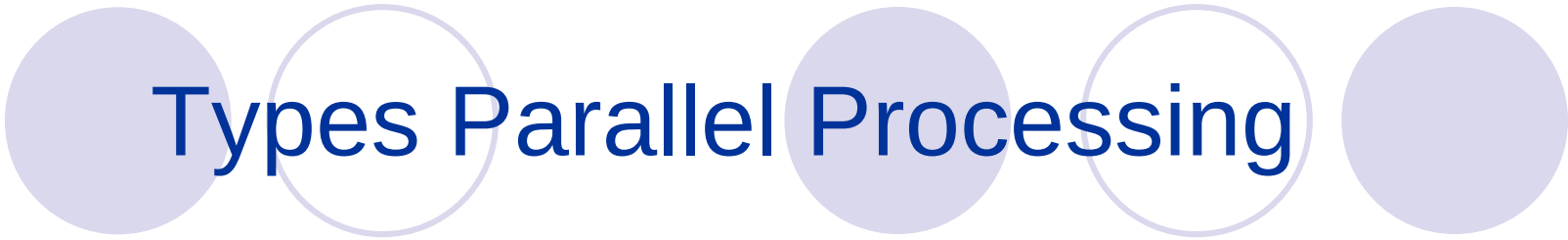
- **Increasingly attractive**
 - • Economics, technology, architecture, application demand
- **Increasingly central and mainstream**
- **Parallelism exploited at many levels**
 - • Instruction-level parallelism
 - • Thread-level parallelism within a microprocessor
 - • Multiprocessor servers
 - • Large-scale multiprocessors (“MPPs”)
- **Same story from memory system perspective**
 - • Increase bandwidth, reduce average latency with many local memories
- **Wide range of parallel architectures make sense**
 - • Different cost, performance and scalability



PART II Different Models

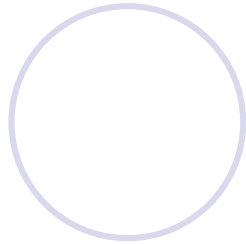
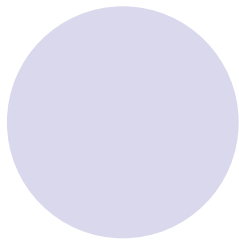
What is Parallel Computing?

- **Parallel computing:** using multiple processors in parallel to solve problems more quickly than with a single processor
- Examples of parallel machines:
 - A **cluster computer** that contains multiple PCs combined together with a high speed network
 - A **shared memory multiprocessor** (SMP*) by connecting multiple processors to a single memory system
 - A **Chip Multi-Processor** (CMP) contains multiple processors (called cores) on a single chip
- Concurrent execution comes from desire for performance; unlike the inherent concurrency in a multi-user distributed system
- * Technically, SMP stands for “Symmetric Multi-Processor”

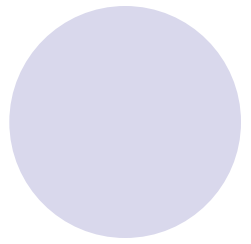
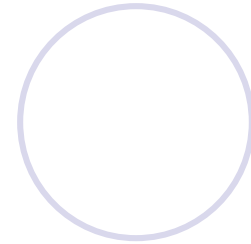
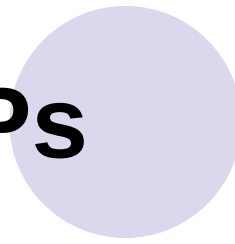


Types Parallel Processing

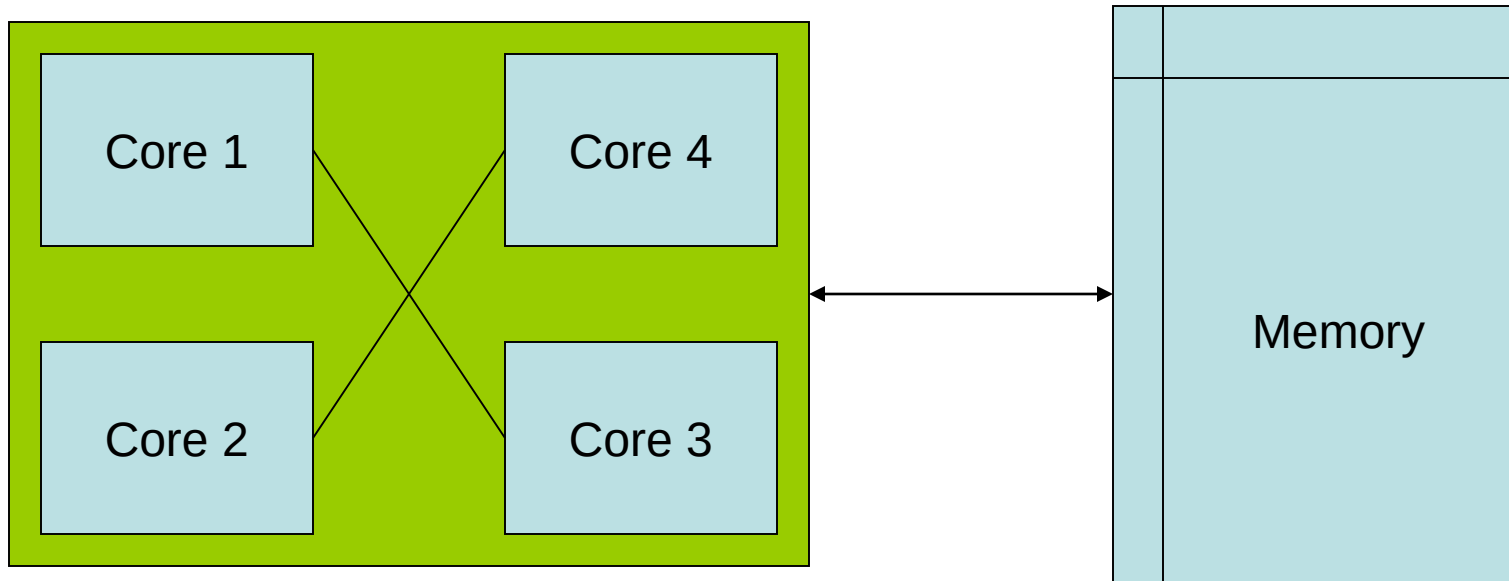
- Chip Multi-Processors (CMP)
- The Shared-Memory Model (***overview***)
- Message Passing Model (***overview***)
- Transaction Model (***overview***)
- The Best Programming Model?



CMPs



● Chip Multi-Processor



The Shared Memory Model

- Typically written using OpenMP, CELL(IBM), Da-Vinci(Texas)
- Software constructs that allow individual processes to physically share certain portions of the same address space
- Seems intuitive for use in CMPs (physical memory chips are shared by the cores)

The Shared Memory Model

● Pros

- Easy to write
- Communication coordination between processes is built-in
- Allows support of both sequential and parallel processes
- Easily scalable to a certain point

● Cons

- Not very general, geared toward loop-level parallelism
- Does not support asynchronous events very well
- Not scalable to distributed systems easily

The Message Passing Model

- **Often written in MPI**
- **Allows communication between processes (threads) using specific message-passing system calls.**
- **All shared data is communicated through messages**
- **Physical memory not necessarily shared**

The Message Passing Model

● Pros

- Allows for asynchronous events
- Does not require programmer to write in terms of loop-level parallelism
- Operates on CMPs AND is scalable to distributed systems
- A more general model of programming...extremely flexible

● Cons

- Considered extremely difficult to write
- Difficult to incrementally increase parallelism
- No implicitly shared data (allowed in MPI 2.0)



The Transaction Model

- Instructions are grouped into sets of transactions
- All-or-nothing model of execution and completion...Atomicity
- Suitable for certain types of applications (ATMs, bank processing, database applications)
- Scalability!

The Transaction Model

● Pros

- Scalable to large distributed systems
- Applicable to a wide range of consumer-oriented applications
- Does not necessarily imply a message-passing or shared-memory interface
- Applicable to many hardware models (assuming support for atomicity)

● Cons

- Not obviously amenable to all problems
- Difficulty to reason about for many applications

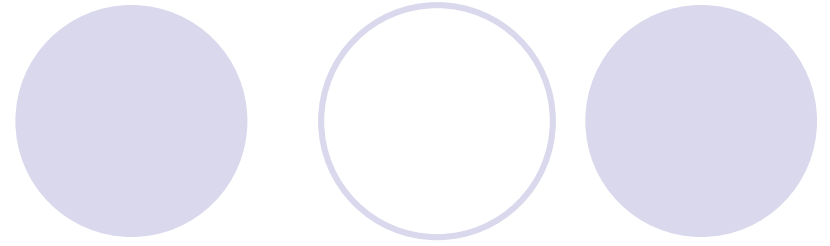
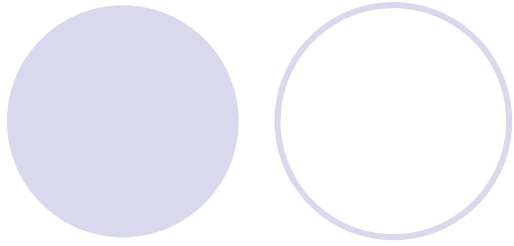
The Best Programming Model?

- The Billion \$\$\$ Question...
- No great general model...(yet)
- Hardware and software issues
- “The vast majority of programmers today don’t grok concurrency, just as the vast majority of programmers 15 years ago didn’t yet grok objects.” (1)
- Economic & Cultural Forces



Conclusion

- **Concurrency in software is an undeniable next step**
- **Different platforms used in different architecture.**
- **How to closely tie the architecture, software and ease of programming?**
- **If we are to continue to exploit processing power to meet the demands of society, we must find a way to utilize it.**



Finish Lecture one on Introduction